

Bloque 05 - Pandas: manipulación de datos

Propósito

Preparar datos tabulares con trazabilidad. Pandas no “arregla” datos por sí solo: permite expresar y validar decisiones sobre ellos.

Lecciones

1. DataFrames, importación y perfilado
2. Selección, tipos y limpieza
3. Columnas derivadas y agregaciones
4. Uniones y cardinalidad
5. Validación y trazabilidad
6. Caso integrado de pedidos

Práctica

Resuelve [la limpieza de pedidos](#) y después consulta [la solución razonada](#).

DataFrames, importación y perfilado

Objetivos y prerequisites

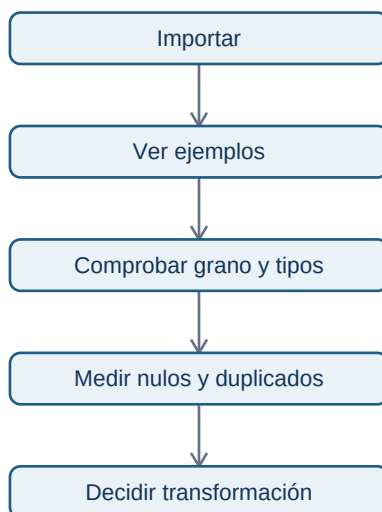
Sabrás abrir una tabla con Pandas e inspeccionarla antes de modificarla. Requiere los bloques de datos y Python.

Un **DataFrame** es una tabla en memoria: filas (observaciones) y columnas (variables) con nombres. Un archivo CSV puede guardar una tabla, pero abrirlo no garantiza que cada columna tenga el tipo ni el significado esperado.

```
import pandas as pd
pedidos = pd.read_csv("pedidos.csv")
pedidos.head()
pedidos.info()
```

`head()` muestra ejemplos; `info()` enseña número de filas, columnas, tipos y valores no nulos. Complementalos con `describe()`, revisión de categorías y comprobación del grano: ¿una fila representa un pedido, una línea de pedido o un usuario?

Este flujo responde a “¿qué debe ocurrir antes de calcular?”



Un error habitual es llamar “ventas” a una columna sin verificar moneda, impuestos o devoluciones. El perfilado abre preguntas; no las responde automáticamente.

Sigue con [selección](#), [tipos](#) y [limpieza](#).

Selección, tipos y limpieza

Objetivos y prerequisites

Seleccionarás columnas y filas, convertirás tipos explícitamente y tratarás problemas sin borrar información a ciegas.

Seleccionar una columna responde una pregunta concreta: `pedidos["importe"]`. Filtrar filas aplica un criterio visible: `pedidos[pedidos["estado"] == "pagado"]`. Antes de filtrar, cuenta qué se excluye y por qué; “pagado” puede ser una definición distinta a “pedido creado”.

Los valores importados como texto requieren conversión controlada:

```
pedidos["importe"] = pd.to_numeric(pedidos["importe"], errors="coerce")
pedidos["fecha"] = pd.to_datetime(pedidos["fecha"], errors="coerce")
```

coerce convierte valores inválidos en ausentes. Es útil porque no inventa una cifra, pero obliga a medir y decidir qué hacer con esos ausentes. No elimines nulos por costumbre: pueden concentrarse en un canal y sesgar el resultado.

Resumen

Limpiar es convertir una regla de calidad en código verificable. Continúa con [transformación y agregación](#).

Columnas derivadas y agregaciones

Objetivos y prerequisites

Crearás medidas derivadas y resumirás una tabla sin perder de vista el grano.

Una columna derivada expresa una regla: `importe_neto = importe - descuento`. Documenta si el descuento ya incluye impuestos y qué sucede cuando falta. `groupby` agrupa filas y aplica una agregación:

```
ventas_canal = pedidos.groupby("canal", as_index=False).agg(
    pedidos=("pedido_id", "nunique"),
    ingresos=("importe_neto", "sum")
)
```

`nunique` cuenta identificadores únicos; `count` cuenta valores no nulos. Elegir uno u otro cambia la métrica. Un promedio de importe también puede ocultar la distribución, por lo que conviene acompañarlo de volumen y percentiles cuando la decisión lo requiera.

Límite

Agregar convierte muchas filas en pocas. Es útil para comparar canales, pero puede ocultar diferencias por país, dispositivo o periodo. Conserva la tabla de origen y registra cada agregación.

Sigue con [uniones y cardinalidad](#).

Uniones y cardinalidad

Objetivos y prerequisites

Combinarás tablas comprobando qué clave conecta las filas y cuántas coincidencias son válidas.

Una unión (merge) cruza dos tablas mediante una **clave**, por ejemplo `cliente_id`. Antes de ejecutarla declara la cardinalidad: uno a uno, uno a muchos o muchos a uno. Muchos a muchos puede ser correcto, pero multiplica combinaciones y requiere una justificación explícita.

```
pedidos_con_clientes = pedidos.merge(
    clientes, on="cliente_id", how="left", validate="many_to_one"
)
```

`validate="many_to_one"` convierte un supuesto en una comprobación: muchos pedidos pueden corresponder a un cliente, pero cada pedido no debe encontrar dos fichas de cliente. Tras unir, compara filas, claves sin coincidencia y totales monetarios.

Error habitual

Ver una columna nueva y asumir que la unión funcionó. Si `clientes` tiene duplicados por error, cada pedido puede repetirse y los ingresos se inflan. La sintaxis válida no demuestra una relación válida.

Continúa con [validación y trazabilidad](#).

Validación y trazabilidad

Objetivos y prerequisites

Definirás controles simples que convierten una transformación en un paso revisable.

Tras cada paso relevante guarda observaciones: número de filas, claves únicas, porcentaje de nulos y totales de negocio. Una validación puede ser una aserción:

```
assert pedidos["pedido_id"].is_unique
assert pedidos["importe_netto"].notna().all()
```

No uses una aserción para ocultar un problema. Si falla, inspecciona los registros y decide si el supuesto era incorrecto o si hay un defecto de datos. Registra filtros, versión de la fuente y fecha de extracción: ese rastro permite reproducir el análisis.

Resumen

Validar no es un último adorno; acompaña a cada transformación. Aplica ahora todo el ciclo en el [caso integrado](#).

Caso integrado de pedidos

Objetivos y prerequisites

Usarás el flujo completo para responder una pregunta simple: “¿qué canal aporta ingresos netos y cuántos pedidos válidos hay?”.

Primero formula el contrato: una fila es un pedido; solo se incluyen estados pagados; importe neto excluye descuento; el periodo es el mes analizado. Después perfila, convierte tipos, mide registros inválidos, crea la columna neta y agrupa por canal. Finalmente compara el total agrupado con el total de pedidos filtrados.

El resultado debe incluir una limitación: si hay pedidos devueltos después de la extracción, los ingresos no representan todavía margen final. Ese tipo de frase es parte del análisis, no una excusa.

Resuelve la [limpieza de pedidos](#) y revisa la solución razonada. El siguiente bloque explorará lo que estos resúmenes sugieren, sin convertirlos de inmediato en causalidad.